

LAB 5: Inheritance in C++ - Documentation

In this lab the aim is to implement linear regression in C++ by reimplementing the supervised-learning framework from previous labs in C++, following the design seen in seminar 4.

The classes `Vector`, `Record`, `Dataset` and `Model` had already been implemented. Our task was to implement the remaining four classes (`Algorithm`, `GradientDescent`, `StochasticGradientDescent` and `SupervisedLearner`) and to update the main function in `TestLearner.cpp` to test the classes.

Overall, our program is able to store datasets, compute gradients for linear regression, learn model using either full-batch gradient descent or stochastic gradient descent, and produce predictions.

Classes, attributes, and methods

Provided classes

1. **Vector**: represents a vector of doubles and can do a set of basic operations.
2. **Record**: pairs an input vector with a scalar output.
3. **Dataset**: represents a collection of records of fixed input dimension.
4. **Model**: represents a linear regression model

New classes implemented

5. **abstract Algorithm**: Represents any supervised learning algorithm
 - **Attributes**:
 - `double learningRate`
 - **Methods**:
 - `Algorithm(double lr)`: constructor
 - `getLearningRate()` → `double`: getter
 - `abstract solve(Dataset &ds)` → `Model`
6. **GradientDescent** extends Algorithm: implements full-batch gradient descent (GD)
 - **Attributes**:
 - `double stoppingCriterion`
 - **Methods**:
 - `GradientDescent(double lr, double sc)`: constructor
 - `getStoppingCriterion()` → `double`: getter
 - `gradient(Dataset &ds, Model &m)` → `Vector`: computes the gradient vector
 - `solve(Dataset &ds)` → `Model`: Performs gradient descent by repeatedly computing the gradient and updating the model until convergence, and returns the learned Model
7. **StochasticGradientDescent** extends Algorithm: implements a smaller-batch GD (SGD)
 - **Attributes**:
 - `int batchsize`
 - `int iterations`
 - **Methods**:
 - `StochasticGradientDescent(double lr, int bs, int i)`: constructor
 - `stochasticGradient(Dataset &ds, Model &m)` → `Vector`: Samples a minibatch of records using and computes the average gradient
 - `solve(Dataset &ds)` → `Model`: Runs SGD for a fixed number of iterations, updating the Model after each minibatch. Returns the learned Model.

8. SupervisedLearner

- **Attributes:**

- *Algorithm* algorithm*: pointer to an Algorithm
- *Dataset dataset*
- *Model* model*: smart pointer to a Model (`std::unique_ptr<Model> model;`)

- **Methods:**

- *SupervisedLearner(Algorithm* a, Dataset& d)*: constructor
- *getAlgorithm() → Algorithm**
- *getDataset() → Dataset&*
- *getModel() → Model**
- *solve()*
- *predict(Vector v) → double*
- *toString() → String*

9. TestLearner: All components were tested using printed outputs and an error counter:

- Verifying Vector, Record, and Dataset basic operations
- Checking that invalid records (wrong dimension) are rejected by Dataset
- Ensuring gradients are correctly computed for Gradient Descent and SGD
- Verifying that both Gradient Descent and Stochastic Gradient Descent learn similar linear models
- Testing predictions on simple linear datasets

Before coding, we discussed several possible designs.

First of all, for the class Algorithm was implemented as an abstract class with:

virtual Model solve(Dataset &ds) = 0; the keyword *virtual* is required in the base class so subclasses can override *solve()*. We intentionally did not include a virtual destructor because the learner never deletes algorithms through a base pointer, making it unnecessary.

For GradientDescent and StochasticGradientDescent, we considered two sampling strategies for SGD: manually with a shuffled index list or using the function `std::sample`. We followed the handout suggestion and used the exact snippet provided, which provides a mini-batch without replacement.

For SupervisedLearner, we discussed different options for storing the learned model: directly as an object, via manual dynamic allocation, or using smart pointers. We chose the latter, implementing it as *std::unique_ptr<Model> model;* because it expresses clear ownership (the learner owns its model), and no manual delete is needed.

Also, the *predict()* method was simplified for C++, since the Lab 5 Dataset no longer performs transformations. The learner augments the input vector and uses the model directly.

This lab applied several object-oriented programming concepts:

- **Abstraction:** Algorithm provides a unified interface for optimization.
- **Inheritance:** GD and SGD extend Algorithm.
- **Polymorphism:** SupervisedLearner interacts with Algorithm through a base pointer.
- **Encapsulation:** All attributes are private.

In practice, our implementation worked as expected. *TestLearner* confirmed that all translated classes worked exactly as Lab 4's, and integrated perfectly with the given ones.

The main challenge was adapting Java semantics to C++, especially using initializer lists for constructors, implementing polymorphism through base-class pointers, or ensuring correct memory management with smart pointers. Luckily, the parameters for *learningRate*, *batchSize* and *iterations* that we chose in Lab4 still worked perfectly.

After working on these issues, the final solution passed all our tests with 0 errors. Overall, this lab helped us experience with C++ abstraction, inheritance, polymorphism, and smart pointers, while teaching us how to translate programs from one programming language to another.